STUDENT

S-0004-TDD

TENTAMEN

250826 DT141G DT201G DT202G Sundsvall

Kurskod	--
Bedömningsform	--
Starttid	26.08.2025 08:00
Sluttid	26.08.2025 13:00
Bedömningsfrist	--
PDF skapad	05.11.2025 13:19
Skapad av	Liselott Engelbrekts

i

Mittuniversitetet
MID SWEDEN UNIVERSITY

Tentamen

DT141G Operativsystemsteori
DT201G, DT202G Operativsystem

Jimmy Åhlander

Instruktioner

Tid	5 timmar
Resurser	Dator och mus. Kalkylator, kurslitteratur, lexikon och formelsamling samt egna anteckningar, penna och sudd. Läsplatta*.
Poäng	100
Krav för godkänt	För godkänt betyg på tentamen fordras betyg E eller högre.
Preliminära betygsgränser	A 90 % B 80 % C 70 % D 60 % E 50 %
Allmänt	Använd stycken, punktlistor och annan textformatering där det är lämpligt när du besvarar frågor med text. Om frågan kräver att du motiverar ditt svar, visar dina beräkningar eller ritar en figur erhålles i normalfallet inga poäng för endast svaret. Ibland är det emellertid endast en rekommendation och inte ett krav att svara på ett visst sätt. Exempel på krav: "*Rita en figur som styrker ditt svar*", "*Svara i bytes*", "*Visa dina beräkningar*". Exempel på rekommendationer: "*Rita en figur om du anser att det styrker ditt svar*", "*Tips: Skapa en tabell*". Tentan är anonymiserad. Skriv aldrig ditt namn, personnummer eller annan identifierande information i dina svar. Vid tekniska problem, kontakta alltid tentamensvakt. Stäng inte av eller starta om datorn utan uppsikt, om du inte redan lämnat in tentamen. *Det är inte tillåtet att använda eller ha tillgång till mobiltelefon, surfplatta eller liknande enheter under

tentamen.
Endast läsplatta *utan tillgång till Internet*, av följande märke är tillåten: ReMarkable.

InsperaScan används ej för denna tentamen.

Mittuniversitetet
Institutionen för data- och elektroteknik (DET)

Jimmy Åhlander, 010 - 142 86 90

1(a) Tänk dig en textfil som består av 3072 byte data, motsvarande 3072 ASCII-tecken.

Anta att filen lagras i ett filsystem av typen NTFS där du granskar filens egenskaper genom utforskaren i Windows (explorer.exe). *Anta att du har standardinställningarna (default settings) för filsystemet*.

Observera att enheterna nedan anges och ska tolkas så som de återges i utforskaren.

Hur skulle storleken presenteras för filen?

Svar: 3 KB.

Hur skulle storleken på disk presenteras för filen?

Svar: 4 KB.

Rätt. 2 av 2 poäng.

1(b) I följande bild ses delar av innehållet i en katalogen med hjälp av kommandot **`ls -lia`** i **`Bash`**. Besvara följande frågor utifrån denna information.

```
total 40
4756235 drwxr-xr-x  5 chuck staff 4096 Apr 15 00:25 .
4756236 drwxr-xr-x 18 chuck staff 4096 Apr 14 23:50 ..
4756237 -rw-r--r--  1 chuck staff 3771 Apr 15 00:20 .bashrc
4756240 -rw-r--r--  3 chuck staff   24 Apr 15 00:23 file.txt
4756241 lrwxrwxrwx  1 chuck staff    8 Apr 15 00:27 file_link -> file.txt
4756242 lrwxrwxrwx  1 chuck staff    1 Apr 15 00:29 curr_dir_link -> .
```

Hur skulle filrättigheterna för den nuvarande katalogen uttryckas på numeriskt (oktalt) format?

Svar: 755 .

Vilken grupp tillhör file.txt?

Svar: staff .

Hur många hårda länkar leder till föräldrakatalogen (*från vad du kan se*)?

Svar: 18 stycken.

Hur många symboliska (mjuka) länkar leder till .bashrc (*från vad du kan se*)?

Svar: 0 stycken.

Vilket nummer har inoden för .bashrc?

Svar: 4756237 .

Rätt. 5 av 5 poäng.

1(c) Utgå ifrån tidigare presenterad information för att även besvara följande: Vilka av nedan angivna operationer är möjliga för den angivna gruppen till respektive fil?

Välj ett eller flera alternativ:

- [ ] Ändra filinnehållet i **`file.txt`**.
- [ ] Exekvera **`file.txt`**, t.ex. som ett skalskript.
- [x] Skriva ut innehållet i **`file.txt`**, t.ex. med **`cat`**.
- [ ] Ändra namnet på **`.bashrc`**, t.ex. med **`mv`**.
- [x] Skriva ut innehållet i den nuvarande katalogen, t.ex. med **`ls`**.
- [ ] Exekvera **`file.txt`**, t.ex. som ett skalskript, genom **`file_link`**.
- [ ] Ta bort **`.bashrc`**, t.ex. med **`rm`**.

Tips:

- Filnamn inklusive filändelser säger inget om filformatet.
- 2 av alternativen är korrekta.

Observera:

- Markerade korrekta svar ger poäng.
- Markerade inkorrekta svar ger minuspoäng.
- Omarkerade svar ger alltid noll poäng.
- Minimum för hela deluppgiften är noll poäng.

Rätt. 5 av 5 poäng.

i Sidersättningsalgoritmer tillämpas i operativsystem som nyttjar virtuellt minne och demand paging. Givet tre fysiska ramar, anta att processen P_1 använder sina logiska sidor i en sekvens angiven av en referenssträng där R representerar en läsoperation och W representerar en skrivoperation.

Processen P_1 har redan tre sidor allokerade i det fysiska minnet, se Tabell 1. P_1 behöver nu sidor enligt följande referenssträng: **`W4 R2 W2 R1 W0 R4 R1 W2 R0`**.

Tabell 1. Inverterad sidtabell för systemet.

Ram	Sida	Process	SC bit	Smutsig/mod bit	Valid bit
0	7	P_1	0	0	1
1	1	P_1	1	1	1
2	3	P_1	0	0	1

Bilderna nedan representerar primärminnet uppdelat i sina ramar vid 10 olika tidpunkter: en för utgångsläget plus en för varje kommande sidreferens. Pekaren som leder till huvudet i den cirkulära kön representeras av symbolen **>**.

2(a) Visa hur sidorna i referenssträngen kommer att allokeras till olika ramar om algoritmen **Optimal Page Replacement (OPT)** tillämpas genom att fylla i vilken sida som är allokerad till vilken ram vid varje tidpunkt, det vill säga efter varje referens i referenssträngen.

Endast sidnumret ska anges i respektive fält.

I de fall där det finns flera potentiella ramar som kan likvärdigt offras ska den ram (obs: ram, inte sida) som kommer tidigast i minnet ersättas, det vill säga helst ersätts ram 0, därefter 1, sist 2.

>7	4	4	4	4	4	4	4	2	2
1	1	1	1	1	1	1	1	1	1
3	3	2	2	2	0	0	0	0	0

Rätt. 4 av 4 poäng.

2(b) Hur många sidfel uppstår när OPT tillämpas i det givna scenariot?

Antal sidfel: 4 .

Rätt. 1 av 1 poäng.

2(c) Visa hur sidorna i referenssträngen kommer att allokeras till olika ramar om algoritmen **Enhanced Second Chance (ESC)** tillämpas genom att fylla i vilken sida som är allokerad till vilken ram vid varje tidpunkt, det vill säga efter varje referens i referenssträngen.

Endast sidnumret ska anges i respektive fält.

>7	4	4	4	4	0	0	0	2	2
1	1	1	1	1	1	1	1	1	1
3	3	2	2	2	2	4	4	4	0

Tips:

- Arbeta initialt med penna och papper för att enklare annotera och skissa på en lösning.
- Tänk på utgångsläget som presenteras i systemets sidtabell.

Delvis rätt. 2.55 av 4 poäng.

2(d) Hur många sidfel uppstår när ESC tillämpas i det givna scenariot?

Antal sidfel: 6 .

Rätt. 1 av 1 poäng.

2(e) Anta i följande två deluppgifter att du får arrangera om hänvisningarna i referenssträngen ovan och använda vilken algoritm som helst för sidersättningen (etablerad variant eller egen).

Vad är det lägsta antalet sidfel som kan uppstå?

Antal sidfel: 2 .

Fel. 0 av 2 poäng.

2(f) Vad är det högsta antalet sidfel som kan uppstå?

Antal sidfel: 9 .

Rätt. 2 av 2 poäng.

ℹ Anta att du har ett exotiskt 19-bit operativsystem där kopplingen mellan logiskt och fysiskt minne sker med ett simpelt relocation register i MMU:n. Med andra ord använder systemet sammanhängande allokering och inte paging. Åtkomsttiden till primärminnet har i detta system uppmätts till 50 ns.

3(a) Hur mycket logiskt minne kan en process maximalt använda i detta system? *Svara i KiB. Ignorera eventuella implementationsspecifika begränsningar.*

Svar: 512 KiB.

Rätt. 2 av 2 poäng.

3(b) Vi byter nu från sammanhängande allokering till paging i systemet, med sidstorleken 0,5 KiB. Detta får ses som främst en teoretisk exercis då paging har begränsad nytta i ett 19-bitars system. I vilket fall används ingen translation lookaside buffer (TLB).

Paging som teknik lider generellt av intern fragmentering. Hur mycket minne kan en process maximalt förlora till intern fragmentering i det här systemet? *Anta att processen nyttjar allokerade sidor i så stor utsträckning som möjligt. Svara i bytes.*

Svar: 511 B.

Rätt. 2 av 2 poäng.

3(c) Hur många sidor består nu det logiska minnet av?

Svar: 1024 sidor.

Rätt. 2 av 2 poäng.

3(d) Processorn i systemet har precis genererat adressen $\mathbf{1500_{10}}$. Vad blir sidnumret och förskjutningen för adressen?

Sidnummer: 2 .

Förskjutning: 19141184 .

Tips:

- Du kan svara i valfri talbas av följande: 2, 10, 16 utifrån den du tycker är enklast att arbeta med. Ange dock *inte* talbasen i fälten.

Delvis rätt. 2 av 4 poäng.

3(e) Vad är den huvudsakliga anledning att icke-sammanhängande allokering som paging används?

Välj ett alternativ

- ○ För att kunna använda sidersättningsalgoritmer.
- ○ För att möjliggöra att TLBs kan användas.
- ○ För att sänka (d.v.s. förbättra) åtkomsttiden till primärminnet.
- ◉ För att minimera intern fragmentering.
- ○ För att eliminera extern fragmentering.

Fel. 0 av 3 poäng.

3(f)

Vi vill nu minimera eventuella tidsförluster genom att också introducera en TLB för minneshanteringen. Den effektiva åtkomsttiden med paging och buffert får sammanlagt **inte öka med *mer än* 20 %** jämfört med den ursprungliga åtkomsttiden till primärminnet.

Vi har fått fem olika bufferts att välja mellan för att uppfylla kravet. Snabbare och träffsäkrare TLBs kostar mer varför vi önskar välja en så billig buffert som möjligt som samtidigt uppfyller vårt krav.

Vilken är den billigaste bufferten som uppfyller kravet?

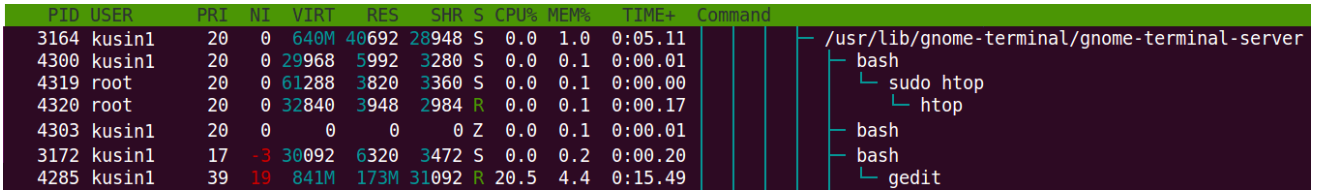
Välj ett alternativ:

- **`billigast:`**
  $85\ \%$ träffchans, $12\ ns$ åtkomsttid
- **`billigare:`**
  $87\ \%$ träffchans, $12\ ns$ åtkomsttid
- **`medeldyr:`**
  $85\ \%$ träffchans, $10\ ns$ åtkomsttid
- **`dyrare:`**
  $90\ \%$ träffchans, $8\ ns$ åtkomsttid
- **`dyrast:`**
  $95\ \%$ träffchans, $5\ ns$ åtkomsttid
- Ingen buffert uppfyller kravet.

Rätt. 5 av 5 poäng.

Din tekniskt obegåvade, hypotetiska kusin som av någon spännande anledning börjat använda Ubuntu har hamnat i knipa. Kusinen arbetade intensivt med ett dokument i **gedit** samtidigt som hen övervakade systemet i **htop**. Enligt utsaga ska kusins katt ha landat på tangentbordet och kommit åt några knappar. Vi kan varken bekräfta eller dementera uppgifterna rörande katten, men något har sannerligen hänt.

Hjälp din kusin att reda ut nedanstående frågor. Du har en skärmdump från **htop** till din hjälp, se nedan.



```
  PID USER      PRI  NI  VIRT   RES   SHR S CPU% MEM%   TIME+  Command
 3164 kusin1     20   0  640M 40692 28948 S  0.0  1.0  0:05.11  ├─ /usr/lib/gnome-terminal/gnome-terminal-server
 4300 kusin1     20   0 29968  5992  3280 S  0.0  0.1  0:00.01  │  ├─ bash
 4319 root       20   0 61288  3820  3360 S  0.0  0.1  0:00.00  │  │  └─ sudo htop
 4320 root       20   0 32840  3948  2984 R  0.0  0.1  0:00.17  │  │     └─ htop
 4303 kusin1     20   0     0     0     0 Z  0.0  0.1  0:00.01  │  ├─ bash
 3172 kusin1     17  -3 30092  6320  3472 S  0.0  0.2  0:00.20  │  ├─ bash
 4285 kusin1     39  19  841M  173M 31092 R 20.5  4.4  0:15.49  │  │  └─ gedit
```

4(a) En av processerna för programmet Bash har en rödmarkerad siffra: **-3** i en kolumn och talet **17** i en annan kolumn. Förklara hur de siffrorna relaterar till varandra och vad respektive siffra innebär.

Skriv in ditt svar här

Båda dessa har att göra med prioritetsnivå och pga hur htop vill visa prioriteten använder den subrtaktion, men det kanske är lättare att förstå genom att observera:
PRI = 20 + NI

Som vi ser visar htop ungifär samma sak fast istället för att visa varsin del så lägger vi NI på andra sidan som en subtraktion:
17 = 20 + (-3)
17 + 3 = 20

Ord: 64

Besvarad.

4(b) Förklara innebörden av de tre olika processtillstånden som visas för de olika processerna i listan.

Skriv in ditt svar här

s - Står för sleeping och helt enkelt är bara att processen inte gör något just nu.

R - Står för running och menas helt enkelt att processen är just nu aktiv och gör sitt jobb.

Z - Står för Zombie och är ett special tillstånd som upphör när ett program skickar en stop signal till sin förälder process som inte kommit fram, alltså har föräldern blivit suspended eller annat som gör att den inte kan ta emot sitt "barns" stopp signal.

Ord: 82

Besvarad.

ℹ Anta att du har ett program vars uppgift är att iterativt inkrementera värdet etthundrafemtio gånger i en global variabel vars initiala värde är noll.

Istället för att addera värdena iterativt i en tråd har arbetet istället fördelats över 3 olika trådar där vardera tråd fått i uppgift att utföra 50 inkrementeringar på den globala variabeln. Detta med avsikten att fördela arbetet över tre processorkärnor för att öka exekveringshastigheten. Vid exekveringen förväntar du dig ett deterministiskt utfall men får istället, förvirrande nog, olika utdata efter varje körning.

5(a) Vilka av följande värden, om något alls och oavsett sannolikhet, kan den globala variabeln erhålla efter en körning av det problematiska programmet?

Välj ett eller flera alternativ:

- [ ] -4
- [ ] 0
- [ ] 1
- [ ] 2
- [x] 3
- [x] 100
- [x] 101
- [x] 150
- [ ] 200

Observera:

- Markerade korrekta svar ger poäng.
- Markerade inkorrekta svar ger minuspoäng.
- Omarkerade svar ger alltid noll poäng.
- Minimum för hela deluppgiften är noll poäng.

Delvis rätt. 1 av 4 poäng.

5(b) Efter några justeringar till programkoden, där du bland annat implementerat mutex-lås för att skydda den kritiska sektionen så upptäcker du att utfallet nu blir deterministiskt med samma utdata efter varje körning. Du ändrar nu även datorsystemet till ett med fyra processorkärnor istället för tre.

Vilka av följande värden, oavsett sannolikhet, kan den globala variabeln nu erhålla efter en körning av det justerade programmet?

Välj ett eller flera alternativ:

- [ ] -4
- [ ] 0
- [ ] 1
- [ ] 2
- [ ] 4
- [ ] 100
- [ ] 101
- [x] 150
- [ ] 200

Observera:

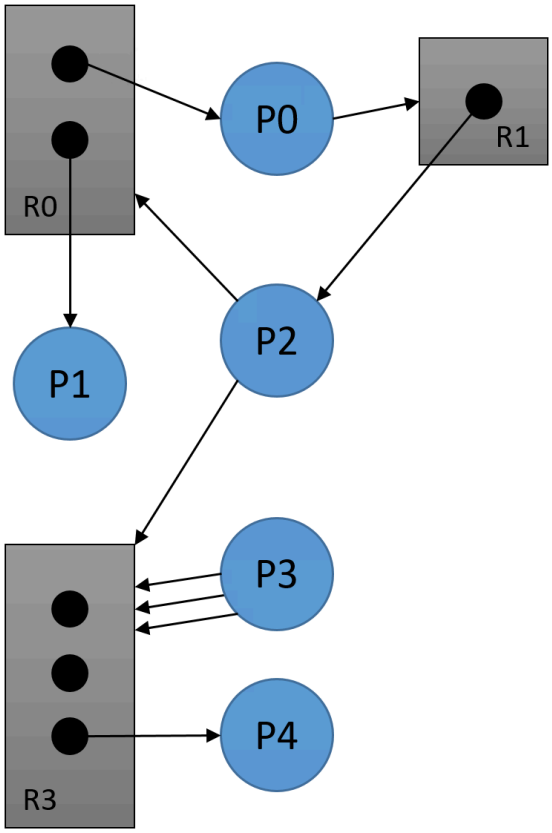
- Markerade korrekta svar ger poäng.
- Markerade inkorrekta svar ger minuspoäng.
- Omarkerade svar ger alltid noll poäng.
- Minimum för hela deluppgiften är noll poäng.

Rätt. 2 av 2 poäng.

ℹ Deadlocks kan uppstå när vissa kriterier är uppfyllda. I ett visst datorsystem råder scenariot som avbildas i resursallokeringsgrafen nedan. Observera att:

- Systemet har ingen preemption eller deadlockförhindrande algoritm.
- I de situationer där flera processer efterfrågar samma resurs samtidigt allokeras resursen slumpartat.
- Processerna kan inte efterfråga fler resurser än de som redan representeras i grafen.
- Samtliga resursinstanser är odelbara.

Besvara följande frågor för det i grafen givna scenariot.



6(a) Har en deadlock uppstått? *Motivera ditt svar*.

Skriv in ditt svar här

Denna kommer inte att resultera i en deadlock och är därmed icket i ett unsafe tillstånd. I det fallet att P4 löst ut sig och P2 tar plats i R3 kommer P3 vänta på P2 för att kunna bli klar, därmed beror allting på P2 att lösa sig vilket den bara gör ifall den också kommer in på R0, vilket den också gör eftersom att P1 bara löser sig vilket P2 väntar in och sedan plaserar sig i R0, vilket då kan lösa kedjan fram till 2 då alla tillstånd nu är lösbara och hela P2 löser sig, då kommer P3 in och löser sig och ingen deadlock har uppstått.

Ord: 110

Besvarad.

6(b) Om en deadlock skulle kunna uppstå i det givna systemet, vilka processer skulle då vara delaktiga i deadlocken?

Välj ett eller flera alternativ:

- [ ] P_0
- [ ] P_1
- [ ] P_2
- [ ] P_3
- [ ] P_4
- [x] Inga processer. En deadlock kan inte uppstå i det givna systemet.

Observera:

- Markerade korrekta svar ger poäng.
- Markerade inkorrekta svar ger minuspoäng.
- Omarkerade svar ger alltid noll poäng.
- Minimum för hela deluppgiften är noll poäng.

Rätt. 3 av 3 poäng.

6(c) Resurs R_3 är en dyrbar resurs där varje aktiv instans ökar underhållskostnaderna för vår organisation. Vi önskar därför rationalisera bort så många instanser av R_3 som möjligt. Vi vill dock inte riskera att systemet hamnar i ett osäkert tillstånd (*unsafe state*) där en deadlock skulle kunna uppstå.

Utan att ändra övriga förutsättningar för scenariot, vad är det lägsta antalet instanser av R_3 som krävs för att systemet ska fungera (om än långsamt) samt vara i ett säkert tillstånd (*safe state*) och därmed helt undvika framtida deadlocks?

Välj ett alternativ

- ( ) 0
- ( ) 1
- ( ) 2
- (•) 3
- ( ) 4
- ( ) 5
- ( ) Fler än 5 resursinstanser av R_3 krävs eftersom systemet för nuvarande är i ett unsafe state på grund av för få resursinstanser av R_3.
- ( ) Det spelar ingen roll hur många resursinstanser av R_3 vi har. Systemet är ändå i ett unsafe state och riskerar försättas i en deadlock.

Rätt. 3 av 3 poäng.

CPU-schemaläggning används för att bestämma vilken process, eller när stöd finns, vilken tråd som ska exekvera på processorn. Trådar ankommer kontinuerligt till en redokö efter ett event eller avslutad I/O-operation.

I följande scenario, vid tidpunkten t har processorn precis blivit tillgänglig. Två trådar har också precis ankommit till redokön i väntan på att bli schemalagda. Med ytterst avancerat estimerande (en spåkula) kan vi även se att fler trådar snart ankommer till kön, enligt nedan angivna ankomsttider.

Tabell. Redokön vid tidpunkten t.

Köplats (#)	Tråd	Uppskattad CPU-sprint (ms)	Ankomsttid (ms)
1	T_1	8	t
2	T_2	4	t
3	T_3	10	t
--	T_4	6	$t+10$

Du kan för dessa uppgifter anta att tidskostnaden för context switching är försumbar. Det är enklast att visualisera sig scenariot om du antar att $t = 0$ vid starten.

7(a) Anta att trådarna schemaläggs med *First-come, First-served (FCFS)*. Vad blir väntetiden för varje tråd och medelväntetiden (tiden varje tråd väntar i snitt)? *Svara exakt*.

T1: 0 ms.

T2: 8 ms.

T3: 12 ms.

T4: 12 ms.

Medelväntetid: 8 ms.

Rätt. 5 av 5 poäng.

7(b) Anta att trådarna schemaläggs med *Round Robin (RR)* där tidskvantumet är $q = 4\ ms$. Vad blir väntetiden för varje tråd och medelväntetiden (tiden varje tråd väntar i snitt)? *Svara exakt*.

T1: 16 ms.

T2: 4 ms.

T3: 20 ms.

T4: 12 ms.

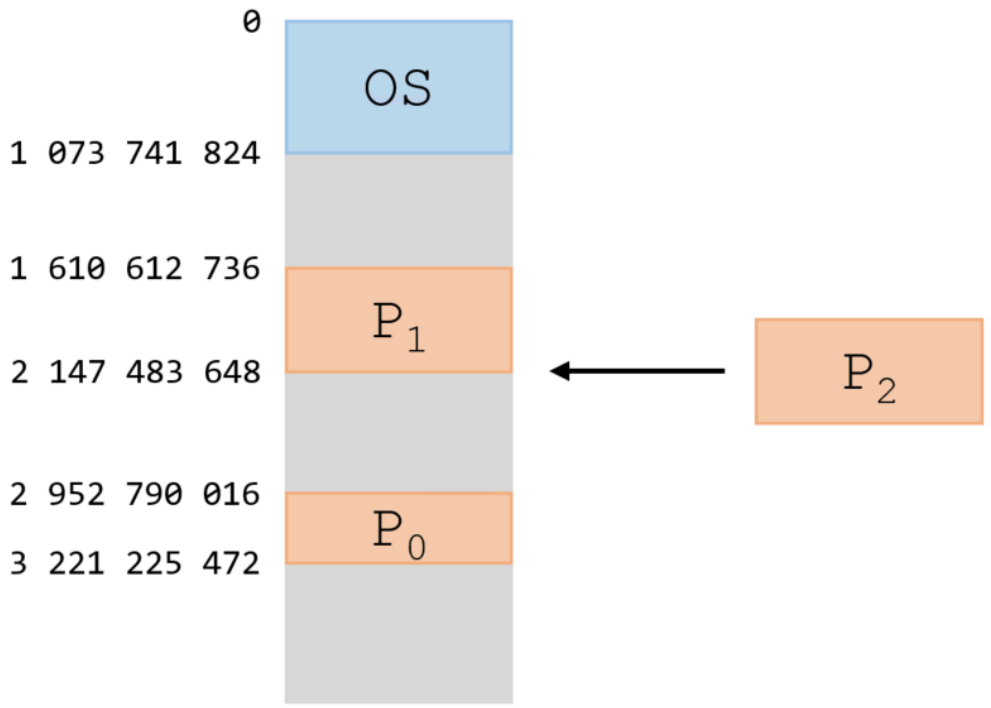
Medelväntetid: 13 ms.

Delvis rätt. 1 av 5 poäng.

Sammanhängande allokering kan användas för att allokera processers minne i enhetliga minnesspann.

Anta att du har ett system med 4 GiB primärminne som allokeras sammanhängande. Operativsystemet och två processer har redan allokerats till minnet i detta system, se figuren nedan. I figuren anges adressen för starten av varje block av minne på decimalform. T.ex. är operativsystemet placerat i spannet 0 till och med 1 073 741 823. Observera att figuren *inte är skalenlig*.

Utrymme behöver nu allokeras för en ny process P_2 som kräver 600 MiB. Besvara följande frågor relaterade till det givna scenariot.



8(a) På vilken adress kommer processens bas placeras om metoden **first-fit** används? *Svara med talbas 10*.

Ange adressen här: 2147483648 .

Rätt. 2 av 2 poäng.

8(b) På vilken adress kommer processens bas placeras om metoden **best-fit** används? *Svara med talbas 10*.

Ange adressen här: 2147483648 .

Rätt. 2 av 2 poäng.

8(c) På vilken adress kommer processens bas placeras om metoden **worst-fit** används? *Svara med talbas 10*.

Ange adressen här: 3221225472 .

Rätt. 2 av 2 poäng.

8(d) Anta nu att du skapar ytterligare processer som alla har samma storlek som P_2. Du upprepar detta tills att det inte längre går att allokera minne till nya sådana processer. Enligt 50-procentregeln, hur mycket minne har nu gått förlorat till extern fragmentering?

Anta att inga processer stängs av och att ingen kompaktering eller swapping sker. Svara i MiB.

Extern fragmentering: 1365 MiB.

Rätt. 2 av 2 poäng.

8(e) 50-procentregeln är endast en tumregel. Hur mycket minne har egentligen gått förlorat till extern fragmentering?

Extern fragmentering: 1104 MiB.

Rätt. 2 av 2 poäng.

9(a) Anta att du har ett filsystem som använder länkad allokering med 64-bitars adresser och en blockstorlek på 128 B.

Vad blir den maximala filstorleken och overheaden för en fil i detta system? *Svaren anges i EiB*.

Maximal filstorlek: 2048 EiB.

Overhead: 64 EiB.

Tips:

- $1\ EiB = 1024\ PiB$
- $1\ PiB = 1024\ TiB$

Delvis rätt. 2 av 4 poäng.

9(b) Anta att du har ett filsystem liknande UFS som använder hierarkisk indexerad allokering, även kallat *combined scheme*, med 32-bitars adresser och en blockstorlek på 32 B. I inoden finns 10 direkta, 1 indirekt och 1 dubbelt indirekt pekare som leder till data- och indexblock.

Vad blir den maximala filstorleken och overheaden i detta system? *Svaren anges i bytes.*

Maximal filstorlek: 2624 B.

Overhead: 736 B.

Delvis rätt. 2 av 4 poäng.

9(c) Du arbetar nu med det tidigare angivna filsystemet med hierarkisk indexerad allokering. Anta att du har möjligheten att ändra adresstorleken. Blockstorleken och övriga egenskaper är oförändrade. Vilken adresstorlek a skulle ge den största möjliga maximala filstorleken?

Du kan anta att adresstorleken a är ett positivt antal bitar: $a \in \mathbb{Z}^+$ som inte nödvändigtvis är en tvåpotens.

Motivera ditt svar.

Skriv in dina beräkningar och ditt svar här

Om man observerar ekvationen för filstorleken kan man bara adera ihop för varsinna pekare därmed:
m = filstorleken
m0 = filstorlek direkt
m1 = filstorlek indirekt
m2 = filstorlek dubbelt indirekt
p = pekare per block
i2 = antal dubbelt indirekta pekare
s2 = adresslängd i byte

m = m0 + m1 + m2
då vi ser att m2 styrs som följande:
m2 = p^2 + p + i2

denna ekvation kan inte sänkas och därmed blir filstorleken itterativt större beroende på a

p blir som störst när s2 är som minst 1

Ord: 92

Besvarad.